



Universidad Nacional de Colombia - sede Bogotá
Facultad de Ingeniería
Departamento de Sistemas e Industrial
Curso: Ingeniería de Software 1 (2016701)

TEST user_services

1. Contexto

Las pruebas cubren tres funcionalidades centrales del sistema: visualización de rutinas del usuario (RF_6), visualización de ejercicios de una rutina (RF_8) y consulta del perfil personal del usuario (RF_9).

Las pruebas fueron implementadas con la herramienta unittest de Python, haciendo uso de unittest.mock para aislar la capa de servicios de la base de datos. Esto garantiza que se valide exclusivamente la lógica de negocio (sentencias if, validaciones, formateo de respuesta) sin depender de Django ni de una conexión real a la BD, cumpliendo con el requisito de no implementar tests de integración.

2. Herramienta y configuración del entorno

2.1 Herramienta utilizada

Se utilizó unittest (librería estándar de Python) junto con unittest.mock para el mocking de dependencias.

Componente	Detalle
Lenguaje	Python 3.14
Framework de testing	unittest (stdlib)
Mocking	unittest.mock — @patch, MagicMock
Coverage	coverage.py v7.14.1
Archivo de tests	gym_app/tests/test_user_services.py

2.2 Comando de ejecución

Desde la carpeta raíz Backend_arquitectura_capas/:

```
python -m unittest gym_app.tests.test_user_services -v
```

Para generar el reporte de cobertura:

```
python -m coverage run -m unittest gym_app.tests.test_user_services -v  
python -m coverage report
```

```
python -m coverage html
```

2.3 Estrategia de aislamiento (sin Django ni BD)

Dado que el proyecto aún no tiene Django configurado ni conexión a base de datos, se bloquearon los módulos de Django y los modelos antes de importar el servicio, usando `sys.modules`. Esto permite correr los tests en cualquier entorno sin necesidad de configuración adicional:

```
sys.modules.setdefault('django', MagicMock())
sys.modules.setdefault('django.db.models', MagicMock())
sys.modules.setdefault('gym_app.models', MagicMock())
sys.modules.setdefault('gym_app.repositories.user_repo', MagicMock())
sys.modules.setdefault('gym_app.repositories.validation_repo', MagicMock())
```

3. Descripción de pruebas unitarias por funcionalidad

RF_6 — get_user_routines_service

- `test_success`: verifica que dado un usuario válido con rutinas, el servicio retorna la lista correcta con nombre de rutina.
- `test_user_with_no_routines`: caso límite — usuario válido pero sin rutinas asignadas, el servicio retorna lista vacía sin lanzar excepción.
- `test_invalid_user_id_negative` / `test_invalid_user_id_zero`: casos límite — IDs inválidos (negativos o cero) lanzan `ValueError` antes de consultar la BD.

RF_8 — get_routine_exercises_service

- `test_success`: verifica que dado un `routine_id` válido, retorna todos los ejercicios con sus series y repeticiones.
- `test_routine_with_no_exercises`: caso límite — rutina existente pero sin ejercicios asignados retorna lista vacía.
- `test_invalid_routine_id_zero`: caso límite — ID cero lanza `ValueError`.

RF_9 — get_user_profile_service

- `test_success_strips_password`: verifica que el campo `password` nunca aparece en la respuesta, independiente de lo que devuelva el repo.
- `test_user_not_found`: caso límite — repo retorna `None`, el servicio lanza `ValueError`.
- `test_invalid_user_id`: caso límite — ID inválido lanza `ValueError` sin llegar al repo.

3.14. Evidencia de ejecución

Las 10 pruebas unitarias fueron ejecutadas exitosamente desde el entorno del proyecto usando el comando `python -m unittest`. A continuación se muestra la evidencia:

Resultado de ejecución (10/10 OK):

```

PS D:\UNAL\Quinto Semestre\IngeSoft\x\Backend_arquitectura_capas> python -m coverage run -m unittest gym_app.tests.test_
user_services -v
test_invalid_routine_id_zero (gym_app.tests.test_user_services.TestGetRoutineExercises.test_invalid_routine_id_zero)
ID cero → ValueError antes de llamar al repo. ... ok
test_routine_with_no_exercises (gym_app.tests.test_user_services.TestGetRoutineExercises.test_routine_with_no_exercises)
Rutina válida pero sin ejercicios → lista vacía, sin excepción. ... ok
test_success (gym_app.tests.test_user_services.TestGetRoutineExercises.test_success)
Camino feliz: rutina con ejercicios asignados. ... ok
test_invalid_user_id (gym_app.tests.test_user_services.TestGetUserProfile.test_invalid_user_id)
ID inválido → ValueError antes de llamar al repo. ... ok
test_success_strips_password (gym_app.tests.test_user_services.TestGetUserProfile.test_success_strips_password)
Perfil encontrado → se retorna sin el campo password. ... ok
test_user_not_found (gym_app.tests.test_user_services.TestGetUserProfile.test_user_not_found)
Repo retorna None → ValueError. ... ok
test_invalid_user_id_negative (gym_app.tests.test_user_services.TestGetUserRoutines.test_invalid_user_id_negative)
ID negativo → ValueError antes de llamar al repo. ... ok
test_invalid_user_id_zero (gym_app.tests.test_user_services.TestGetUserRoutines.test_invalid_user_id_zero)
ID cero → ValueError antes de llamar al repo. ... ok
test_success (gym_app.tests.test_user_services.TestGetUserRoutines.test_success)
Camino feliz: usuario con rutinas asignadas. ... ok
test_user_with_no_routines (gym_app.tests.test_user_services.TestGetUserRoutines.test_user_with_no_routines)
Usuario válido pero sin rutinas → lista vacía, sin excepción. ... ok

-----
Ran 10 tests in 0.004s

OK

```

Reporte de cobertura — vista general:

Coverage report: 61%

Files Functions Classes

coverage.py v7.14.1, created at 2026-06-14 17:04 -0500

File ▲	statements	missing	excluded	coverage
gym_app__init__.py	0	0	0	100%
gym_app\services__init__.py	0	0	0	100%
gym_app\services\user_services.py	122	75	0	39%
gym_app\tests__init__.py	0	0	0	100%
gym_app\tests\ranges_physic_test.py	2	0	0	100%
gym_app\tests\test_user_services.py	72	1	0	99%
Total	196	76	0	61%

coverage.py v7.14.1, created at 2026-06-14 17:04 -0500

Reporte de cobertura — vista por funciones:

Coverage report: 61%

Files

Functions

Classes

coverage.py v7.14.1, created at 2026-06-14 17:04 -0500

File ▲	function	statements	missing	excluded	coverage
gym_app__init__.py	(no function)	0	0	0	100%
gym_app\services__init__.py	(no function)	0	0	0	100%
gym_app\services\user_services.py	empty	1	1	0	0%
gym_app\services\user_services.py	valid_email	2	2	0	0%
gym_app\services\user_services.py	valid_passord	11	11	0	0%
gym_app\services\user_services.py	valid_lenght	1	1	0	0%
gym_app\services\user_services.py	valid_uppercase	1	1	0	0%
gym_app\services\user_services.py	valid_lowercase	1	1	0	0%
gym_app\services\user_services.py	valid_digit	1	1	0	0%
gym_app\services\user_services.py	valid_special_char	2	2	0	0%
gym_app\services\user_services.py	same_passwords	1	1	0	0%
gym_app\services\user_services.py	hash_password	1	1	0	0%
gym_app\services\user_services.py	register_user	13	13	0	0%
gym_app\services\user_services.py	authenticate_user	7	7	0	0%
gym_app\services\user_services.py	validate_age	1	1	0	0%
gym_app\services\user_services.py	validate_weight	1	1	0	0%
gym_app\services\user_services.py	validate_height	1	1	0	0%
gym_app\services\user_services.py	register_initial_data	11	11	0	0%
gym_app\services\user_services.py	update_user_data	19	19	0	0%
gym_app\services\user_services.py	get_user_routines_service	6	0	0	100%
gym_app\services\user_services.py	get_routine_exercises_service	6	0	0	100%
gym_app\services\user_services.py	get_user_profile_service	8	0	0	100%
gym_app\services\user_services.py	(no function)	27	0	0	100%
gym_app\tests__init__.py	(no function)	0	0	0	100%
gym_app\tests\ranges_physic_test.py	(no function)	2	0	0	100%
gym_app\tests\test_user_services.py	TestGetUserRoutines.test_success	6	0	0	100%
gym_app\tests\test_user_services.py	TestGetUserRoutines.test_user_with_no_routines	3	0	0	100%
gym_app\tests\test_user_services.py	TestGetUserRoutines.test_invalid_user_id_negative	2	0	0	100%
gym_app\tests\test_user_services.py	TestGetUserRoutines.test_invalid_user_id_zero	2	0	0	100%
gym_app\tests\test_user_services.py	TestGetRoutineExercises.test_success	7	0	0	100%
gym_app\tests\test_user_services.py	TestGetRoutineExercises.test_routine_with_no_exercises	3	0	0	100%
gym_app\tests\test_user_services.py	TestGetRoutineExercises.test_invalid_routine_id_zero	2	0	0	100%
gym_app\tests\test_user_services.py	TestGetUserProfile.test_success_strips_password	5	0	0	100%
gym_app\tests\test_user_services.py	TestGetUserProfile.test_user_not_found	3	0	0	100%
gym_app\tests\test_user_services.py	TestGetUserProfile.test_invalid_user_id	2	0	0	100%
gym_app\tests\test_user_services.py	(no function)	37	1	0	97%
Total		196	76	0	61%

5. Análisis de cobertura

El reporte de coverage arrojó un 61% de cobertura total sobre los archivos del proyecto. Este porcentaje se explica porque `user_services.py` contiene 122 sentencias en total, de las cuales 75 corresponden a funciones implementadas por otros integrantes del equipo (`register_user`, `authenticate_user`, `update_user_data`, entre otras) que no son objeto de esta entrega.

Las tres funciones bajo mi responsabilidad tienen 100% de cobertura, validando todos los caminos posibles de ejecución: camino feliz, casos límite de entrada y manejo de respuestas vacías o inexistentes del repositorio. El 61% total refleja el estado parcial del proyecto, no una deficiencia en las pruebas implementadas.

TEST `admin_services`:

1. Contexto

Las pruebas cubren dos funcionalidades referentes a el registro de máquinas (RF_1A), y el registro de ejercicios (RF_1B).

Las pruebas fueron implementadas con la herramienta `unittest` de Python, haciendo uso de `unittest.mock` para aislar la capa de servicios de la base de datos. Esto garantiza que se valida exclusivamente la lógica de negocio (sentencias `if`, validaciones, formato de respuesta) sin depender de Django ni de una conexión real a la BD, cumpliendo con el requisito de no implementar tests de integración.

2. Herramienta y configuración del entorno

2.1 Herramienta utilizada

Se utilizó `unittest` (librería estándar de Python) junto con `unittest.mock` para el mocking de dependencias.

Componente	Detalle
Lenguaje	Python 3.12.7
Framework de testing	<code>pytest</code> (stdlib)
Mocking	<code>unittest.mock</code> — <code>@patch</code> , <code>MagicMock</code>
Coverage	<code>coverage.py</code> v7.14.1
Archivo de tests	<code>gym_app/tests/test_user_services.py</code>

2.2 Comando de ejecución

Desde la carpeta raíz `Backend_arquitectura_capas/`:

```
pytest tests/test_admin_services.py --cov=services.admin_services
--cov-report=term-missing -v
```

Para generar el reporte de cobertura:

```
python -m coverage html
```

2.3 Estrategia de aislamiento (sin Django ni BD)

Dado que el proyecto aún no tiene Django configurado ni conexión a base de datos, se bloquearon los módulos de Django y los modelos antes de importar el servicio, usando `sys.modules`. Esto permite correr los tests en cualquier entorno sin necesidad de configuración adicional:

```
sys.modules.setdefault('django', MagicMock())
sys.modules.setdefault('django.db.models', MagicMock())
sys.modules.setdefault('gym_app.models', MagicMock())
sys.modules.setdefault('gym_app.repositories.admin_repo', MagicMock())
```

3. Descripción de pruebas unitarias por funcionalidad

RF_1A — register_machine

- **TestNoInvalidChars**: verifica que el nombre ingresado no contenga caracteres especiales (!@#\$%^&*()+=|{}|\|;:"<>,.?/).
- **TestValidNameLength**: Límites, verifica que el nombre contenga entre 1 y 50 caracteres.
- **TestRegisterMachine**: Prueba integral de la función **register_machine**.

RF_1B — register_exercise

- **TestNoInvalidChars**.
- **TestValidNameLength**.
- **TestCorrectDescriptionLength**: Verifica que la descripción se encuentre por debajo de los 500 caracteres.
- **TestRegisterExercise**: Prueba integral de la función **register_exercise**.

3.14. Evidencia de ejecución

Las 11 pruebas unitarias fueron ejecutadas exitosamente desde el entorno del proyecto.

A continuación se muestra la evidencia:

Resultado de ejecución (10/10 OK):

```
>> pytest tests/test_admin_services.py --cov=services.admin_services --cov-report=term-missing -v
===== test session starts =====
platform win32 -- Python 3.12.7, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\nickc\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\nickc\OneDrive\Desktop\gym_app
plugins: cov-7.1.0
collected 27 items

tests/test_admin_services.py::TestNoInvalidChars::test_returns_false_for_invalid_chars PASSED [ 3%]
tests/test_admin_services.py::TestNoInvalidChars::test_returns_true_for_valid_name PASSED [ 7%]
tests/test_admin_services.py::TestValidNameLength::test_returns_false_for_empty_name PASSED [ 11%]
tests/test_admin_services.py::TestValidNameLength::test_returns_false_for_name_too_long PASSED [ 14%]
tests/test_admin_services.py::TestValidNameLength::test_returns_true_for_maximum_length PASSED [ 18%]
tests/test_admin_services.py::TestValidNameLength::test_returns_true_for_minimum_length PASSED [ 22%]
tests/test_admin_services.py::TestCorrectDescriptionLength::test_returns_false_for_description_too_long PASSED [ 25%]
tests/test_admin_services.py::TestCorrectDescriptionLength::test_returns_true_for_empty_description PASSED [ 29%]
tests/test_admin_services.py::TestCorrectDescriptionLength::test_returns_true_for_max_length PASSED [ 33%]
tests/test_admin_services.py::TestCorrectDescriptionLength::test_returns_true_for_valid_description PASSED [ 37%]
tests/test_admin_services.py::TestRegisterMachine::test_name_is_saved_in_uppercase PASSED [ 40%]
tests/test_admin_services.py::TestRegisterMachine::test_raises_if_invalid_area PASSED [ 44%]
tests/test_admin_services.py::TestRegisterMachine::test_raises_if_invalid_chars_in_name PASSED [ 48%]
tests/test_admin_services.py::TestRegisterMachine::test_raises_if_machine_already_exists PASSED [ 51%]
tests/test_admin_services.py::TestRegisterMachine::test_raises_if_name_is_empty PASSED [ 55%]
tests/test_admin_services.py::TestRegisterMachine::test_raises_if_name_too_long PASSED [ 59%]
tests/test_admin_services.py::TestRegisterMachine::test_registers_machine_successfully PASSED [ 62%]
tests/test_admin_services.py::TestRegisterExercise::test_name_is_saved_in_uppercase PASSED [ 66%]
tests/test_admin_services.py::TestRegisterExercise::test_raises_if_description_too_long PASSED [ 70%]
tests/test_admin_services.py::TestRegisterExercise::test_raises_if_exercise_already_exists PASSED [ 74%]
tests/test_admin_services.py::TestRegisterExercise::test_raises_if_invalid_area PASSED [ 77%]
tests/test_admin_services.py::TestRegisterExercise::test_raises_if_invalid_chars_in_name PASSED [ 81%]
tests/test_admin_services.py::TestRegisterExercise::test_raises_if_name_too_long PASSED [ 85%]
tests/test_admin_services.py::TestRegisterExercise::test_registers_exercise_successfully PASSED [ 88%]
tests/test_admin_services.py::TestGetAllMachines::test_returns_all_machines PASSED [ 92%]
tests/test_admin_services.py::TestGetMachineById::test_returns_machine_by_id PASSED [ 96%]
tests/test_admin_services.py::TestGetExercisesByMachine::test_returns_exercises_by_machine PASSED [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.12.7-final-0 -----

Name                               Stmts  Miss  Cover   Missing
-----
services\admin_services.py         40      0  100%
TOTAL                             40      0  100%

===== 27 passed in 0.12s =====
PS C:\Users\nickc\OneDrive\Desktop\gym_app>
```

Reporte de cobertura — vista general:

Coverage report: 100%

Files Functions Classes

coverage.py v7.14.1, created at 2026-06-14 19:22 -0500

File ▲	statements	missing	excluded	coverage
services\admin_services.py	40	0	0	100%
Total	40	0	0	100%

coverage.py v7.14.1, created at 2026-06-14 19:22 -0500

Reporte de cobertura — vista por funciones:

Coverage report: 100%

Files Functions Classes

coverage.py v7.14.1, created at 2026-06-14 19:22 -0500

File ▲	function	statements	missing	excluded	coverage
services\admin_services.py	no_invalid_chars	2	0	0	100%
services\admin_services.py	valid_name_length	1	0	0	100%
services\admin_services.py	correct_description_length	1	0	0	100%
services\admin_services.py	register_machine	11	0	0	100%
services\admin_services.py	register_exercise	13	0	0	100%
services\admin_services.py	get_all_machines	1	0	0	100%
services\admin_services.py	get_machine_by_id	1	0	0	100%
services\admin_services.py	get_exercises_by_machine	1	0	0	100%
services\admin_services.py	(no function)	9	0	0	100%
Total		40	0	0	100%

coverage.py v7.14.1, created at 2026-06-14 19:22 -0500

5. Análisis de cobertura

Dado que se abarcaron funciones muy concretas se pudo alcanzar un 100% de la cobertura.

TEST user_services_part_2

1. Contexto

Las pruebas cubren tres funcionalidades centrales del sistema: creación y autenticación de cuenta de usuario (RF_2), registro de datos de las pruebas físicas del usuario (RF_3), actualización de los datos de las pruebas físicas del usuario (RF_7).

Las pruebas fueron implementadas con la herramienta unittest de Python, haciendo uso de unittest.mock para aislar la capa de servicios de la base de datos. Esto garantiza que se valida exclusivamente la lógica de negocio (sentencias if, validaciones, formateo de respuesta) sin depender de Django ni de una conexión real a la BD, cumpliendo con el requisito de no implementar tests de integración.

2. Herramienta y configuración del entorno

2.1 Herramienta utilizada

Se utilizó unittest (librería estándar de Python) junto con unittest.mock para el mocking de dependencias.

Componente	Detalle
Lenguaje	Python 3.14
Framework de testing	unittest (stdlib)
Mocking	unittest.mock — @patch, MagicMock
Coverage	coverage.py v7.14.1
Archivo de tests	gym_app/tests/test_user_services.py

2.2 Comando de ejecución

Desde la carpeta raíz Backend_arquitectura_capas/:

```
python -m unittest gym_app.tests.test_user_services -v
```

Para generar el reporte de cobertura:

```
python -m coverage run -m unittest gym_app.tests.test_user_services -v
```

```
python -m coverage report
```

```
python -m coverage html
```

Aclaración importante:

Debido a la forma en que se maneja la referenciación de las distintas funciones del programa aparecían en coverage parte de modelos y de repositorios, sin embargo estas funciones únicamente son referenciando a la función en sí a la hora de la búsqueda más no las ejecuta ya que los mocks se encargan de eso

2.3 Estrategia de aislamiento (sin Django ni BD)

Dado que el proyecto aún no tiene Django configurado ni conexión a base de datos, se bloquearon los módulos de Django y los modelos antes de importar el servicio, usando sys.modules. Esto permite correr los tests en cualquier entorno sin necesidad de configuración adicional:

```
sys.modules['django'] = MagicMock()
sys.modules['django.db'] = MagicMock()
sys.modules['django.contrib.auth.models'] = MagicMock()
import gym_app.services.user_services as user_services
import gym_app.tests.ranges_physic_test as ranges
```

3. Descripción de pruebas unitarias por funcionalidad

RF_2 — register_user y authenticate_user

- **test_empty_fields:** Evalúa combinaciones de campos vacíos en el registro para asegurar que falte información crítica antes de procesar.
- **test_valid_email_boundaries:** Caso límite — Evalúa estructuras fronterizas de correos electrónicos (longitud mínima válida, subdominios y formatos inválidos sin TLD o locales).
- **test_password_security_rules:** Prueba de forma exhaustiva las reglas complejas de la contraseña, validando el caso límite de exactamente 8 caracteres y fallos por ausencia de mayúsculas, minúsculas, dígitos o caracteres especiales.
- **test_validate_age_boundaries / test_validate_weight_boundaries:** Casos límite — Verifican el comportamiento del sistema exactamente en las fronteras de los rangos realistas (12 a 100 años; 30 a 250 kg) tanto para valores válidos como para desviaciones decimales inválidas.
- **test_validate_age_invalid_returns_true/test_validate_weight_invalid_returns_true:** Verifican que cualquier valor de edad o peso fuera de los rangos permitidos active correctamente la alerta retornando True.
- **test_register_user_success:** Verifica el registro correcto de un usuario y el procesamiento seguro de la clave mediante el algoritmo de hasheo SHA256 antes de interactuar con el repositorio.
- **test_register_user_invalid_email_raises:** Verifica que un correo con estructura inválida lance un ValueError y aborte inmediatamente la persistencia en la base de datos.
- **test_register_user_empty_fields_raises:** Verifica que la presencia de campos vacíos lance un ValueError con el mensaje correspondiente, asegurando que no se intente guardar el registro.
- **test_register_user_password_mismatch_raises:** Verifica que si la contraseña y su confirmación no coinciden, se lance un ValueError y se bloquee el almacenamiento.
- **test_register_user_invalid_password_raises:** Verifica que una contraseña que no cumple con las directrices de complejidad o longitud mínima lance un ValueError.

- `test_authenticate_user_wrong_password`: Verifica que un usuario existente en el sistema sea rechazado con un `ValueError` si la contraseña provista no coincide con el hash almacenado.
- `test_authenticate_user_success_returns_user`: Verifica que el servicio de autenticación retorne exitosamente el objeto de usuario completo cuando las credenciales ingresadas son correctas.
- `test_authenticate_user_not_found_raises`: Caso límite — Verifica que si el repositorio no encuentra el correo electrónico, el servicio lance un `ValueError` indicando que el usuario no existe.

RF_3 — register_initial_data

- `test_register_initial_data_invalid_goal_fails`: Caso límite — Verifica que si el identificador del objetivo de entrenamiento (`goal_id`) no existe en la base de datos, el servicio aborta lanzando un `ValueError`.
- `test_register_initial_data_invalid_age_raises` / `test_register_initial_data_invalid_weight_raises`: Verifican que el flujo de creación inicial se interrumpa con un `ValueError` si la edad o el peso enviados no superan los filtros de rangos realistas.
- `test_register_initial_data_success`: Verifica el flujo correcto de creación; asegura que se guarde el perfil biométrico y que el ID autogenerado por el ORM sea heredado correctamente para registrar el test físico correspondiente.

RF_7 — update_user_data

- `test_update_user_data_invalid_age_raises` / `test_update_user_data_invalid_weight_raises`: Verifican que el servicio de actualización lance un `ValueError` si las nuevas modificaciones de edad o peso violan los rangos realistas admitidos.
- `test_update_initial_data_partial_fill_success`: Caso límite — Se envían algunos campos en `None`. Verifica la capacidad del servicio para recuperar los valores históricos del perfil y del test físico en la base de datos, combinándolos con los nuevos cambios de forma exitosa sin corromper la información previa.
- `test_update_initial_data_invalid_goal_fails`: Caso límite — Verifica que si durante una actualización parcial el `goal_id` (nuevo o recuperado del historial) es inválido o inexistente en la BD, el proceso falle lanzando un `ValueError`.

3.14. Evidencia de ejecución

Las 20 pruebas unitarias fueron ejecutadas exitosamente desde el entorno del proyecto usando el comando `python -m unittest`. A continuación se muestra la evidencia: en adición a los tests que ya estaban anteriormente

Resultado de ejecución (20/20 OK):

```
test_authenticate_user_not_found_raises (gym_app.tests.test_user_services.TestUserServices.test_authenticate_user_not_found_raises)
Verifica que un usuario inexistente lance ValueError. ... ok
test_authenticate_user_success_returns_user (gym_app.tests.test_user_services.TestUserServices.test_authenticate_user_success_returns_user)
Verifica que authenticate_user retorne el usuario cuando la contraseña es correcta. ... Usuario athlete1@test.com autenticado con éxito.
ok
test_authenticate_user_wrong_password (gym_app.tests.test_user_services.TestUserServices.test_authenticate_user_wrong_password)
Verifica que un usuario existente con clave errónea sea rechazado. ... ok
test_empty_fields (gym_app.tests.test_user_services.TestUserServices.test_empty_fields)
Evalúa combinaciones de campos vacíos en el registro. ... ok
test_get_routine_exercises_invalid_id (gym_app.tests.test_user_services.TestUserServices.test_get_routine_exercises_invalid_id)
Edge Case: Invalid routine ID. ... ok
test_get_routine_exercises_success (gym_app.tests.test_user_services.TestUserServices.test_get_routine_exercises_success)
Edge Case: Valid routine. ... ok
test_get_user_profile_not_found (gym_app.tests.test_user_services.TestUserServices.test_get_user_profile_not_found)
Edge Case: User does not exist. ... ok
test_get_user_profile_success_strips_password (gym_app.tests.test_user_services.TestUserServices.test_get_user_profile_success_strips_password)
Edge Case: Ensure sensitive data (password) is removed. ... ok
test_get_user_routines_empty (gym_app.tests.test_user_services.TestUserServices.test_get_user_routines_empty)
Edge Case: Valid user but has no routines assigned. ... ok
test_get_user_routines_invalid_id (gym_app.tests.test_user_services.TestUserServices.test_get_user_routines_invalid_id)
Edge Case: Invalid ID type or value. ... ok
test_get_user_routines_success (gym_app.tests.test_user_services.TestUserServices.test_get_user_routines_success)
Edge Case: Valid user. Returns routine names from Routine table JOIN. ... ok
test_password_security_rules (gym_app.tests.test_user_services.TestUserServices.test_password_security_rules)
Prueba de forma exhaustiva las reglas complejas de la contraseña. ... ok
test_register_initial_data_invalid_age_raises (gym_app.tests.test_user_services.TestUserServices.test_register_initial_data_invalid_age_raises)
Verifica que register_initial_data lance ValueError cuando la edad es inválida. ... ok
test_register_initial_data_invalid_goal_fails (gym_app.tests.test_user_services.TestUserServices.test_register_initial_data_invalid_goal_fails)
Verifica que si el objetivo del plan no se halla en la BD lance error. ... ok
test_register_initial_data_invalid_weight_raises (gym_app.tests.test_user_services.TestUserServices.test_register_initial_data_invalid_weight_raises)
Verifica que register_initial_data lance ValueError cuando el peso es inválido. ... ok
test_register_initial_data_success (gym_app.tests.test_user_services.TestUserServices.test_register_initial_data_success)
Verifica el flujo correcto de registro de datos iniciales. ... Datos iniciales registrados para usuario ID 1.
ok
test_register_user_empty_fields_raises (gym_app.tests.test_user_services.TestUserServices.test_register_user_empty_fields_raises)
Verifica que campos vacíos lancen ValueError y no se intente guardar. ... ok
test_register_user_invalid_email_raises (gym_app.tests.test_user_services.TestUserServices.test_register_user_invalid_email_raises)
Verifica que un correo inválido lance ValueError y no intente guardar. ... ok
test_register_user_invalid_password_raises (gym_app.tests.test_user_services.TestUserServices.test_register_user_invalid_password_raises)
Verifica que una contraseña insegura lance ValueError. ... ok
test_register_user_password_mismatch_raises (gym_app.tests.test_user_services.TestUserServices.test_register_user_password_mismatch_raises)
Verifica que contraseñas distintas lancen ValueError y no se intente guardar. ... ok
test_register_user_success (gym_app.tests.test_user_services.TestUserServices.test_register_user_success)
Verifica registro correcto y hash de clave con algoritmo SHA256. ... Usuario con correo trainer@gym.com registrado con éxito.
ok

test_update_initial_data_invalid_goal_fails (gym_app.tests.test_user_services.TestUserServices.test_update_initial_data_invalid_goal_fails)
Caso de Fallo: la meta (ya sea la nueva o la rescatada) no es válida. ... ok
test_update_initial_data_partial_fill_success (gym_app.tests.test_user_services.TestUserServices.test_update_initial_data_partial_fill_success)
Caso de éxito: Se envían algunos datos en None. ... Datos actualizados registrados para usuario ID 1.
ok
test_update_user_data_invalid_age_raises (gym_app.tests.test_user_services.TestUserServices.test_update_user_data_invalid_age_raises)
Verifica que update_user_data lance ValueError cuando la edad actualizada es inválida. ... ok
test_update_user_data_invalid_weight_raises (gym_app.tests.test_user_services.TestUserServices.test_update_user_data_invalid_weight_raises)
Verifica que update_user_data lance ValueError cuando el peso actualizado es inválido. ... ok
test_valid_email_boundaries (gym_app.tests.test_user_services.TestUserServices.test_valid_email_boundaries)
Evalúa estructuras límites de correos electrónicos. ... ok
test_validate_age_boundaries (gym_app.tests.test_user_services.TestUserServices.test_validate_age_boundaries)
Prueba límites exactos para la edad (12 a 100). ... ok
test_validate_age_invalid_returns_true (gym_app.tests.test_user_services.TestUserServices.test_validate_age_invalid_returns_true)
Verifica que una edad fuera de rango devuelva True. ... ok
test_validate_weight_boundaries (gym_app.tests.test_user_services.TestUserServices.test_validate_weight_boundaries)
Prueba límites exactos para el peso (30 a 250). ... ok
test_validate_weight_invalid_returns_true (gym_app.tests.test_user_services.TestUserServices.test_validate_weight_invalid_returns_true)
Verifica que un peso fuera de rango devuelva True. ... ok
```

Reporte de cobertura — vista general:

File ▲	statements	missing	excluded	coverage
gym_app\models.py	66	11	0	83%
gym_app\repositories__init__.py	0	0	0	100%
gym_app\services__init__.py	0	0	0	100%
gym_app\services\user_services.py	124	2	0	98%
gym_app\tests\ranges_physic_test.py	2	0	0	100%
gym_app\tests\test_user_services.py	252	1	0	99%
Total	444	14	0	97%

Reporte de cobertura — vista por funciones:

Coverage report: 97%

Files

Functions

Classes

coverage.py v7.14.1, created at 2026-06-15 01:47 -0500

File ▲	function	statements	missing	excluded	coverage
gym_app\services\user_services.py	save_user	1	0	0	100%
gym_app\services\user_services.py	save_user_profile	1	0	0	100%
gym_app\services\user_services.py	save_physical_test	1	0	0	100%
gym_app\services\user_services.py	fetch_latest_physical_test	1	0	0	100%
gym_app\services\user_services.py	fetch_user_profile	1	0	0	100%
gym_app\services\user_services.py	fetch_user_by_email	1	0	0	100%
gym_app\services\user_services.py	fetch_goal	1	0	0	100%
gym_app\services\user_services.py	empty	1	0	0	100%
gym_app\services\user_services.py	valid_email	2	0	0	100%
gym_app\services\user_services.py	valid_passord	11	0	0	100%
gym_app\services\user_services.py	valid_lenght	1	0	0	100%
gym_app\services\user_services.py	valid_uppercase	1	0	0	100%
gym_app\services\user_services.py	valid_lowercase	1	0	0	100%
gym_app\services\user_services.py	valid_digit	1	0	0	100%
gym_app\services\user_services.py	valid_special_char	2	0	0	100%
gym_app\services\user_services.py	same_passwords	1	0	0	100%
gym_app\services\user_services.py	hash_password	1	0	0	100%
gym_app\services\user_services.py	register_user	10	0	0	100%
gym_app\services\user_services.py	authenticate_user	7	0	0	100%
gym_app\services\user_services.py	validate_age	1	0	0	100%
gym_app\services\user_services.py	validate_weight	1	0	0	100%
gym_app\services\user_services.py	register_initial_data	9	0	0	100%
gym_app\services\user_services.py	update_user_data	16	0	0	100%
gym_app\services\user_services.py	get_user_routines_service	6	0	0	100%
gym_app\services\user_services.py	get_routine_exercises_service	6	1	0	83%
gym_app\services\user_services.py	get_user_profile_service	8	1	0	88%
gym_app\services\user_services.py	(no function)	31	0	0	100%